

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and communications technology

## Software Requirement Specification

Version 1.2

AIMS – An Internet Media Store

Subject: ITSS Software Development

Group 20

Hồ Ngọc Minh - 20235973

Đồng Đại Huy - 20235945

Ngô Duy An - 20235883

Nguyễn Tuấn Anh - 20235893

Trần Việt Anh - 20226013

*Hanoi, March 2026*

## Table of contents

|                                    |   |
|------------------------------------|---|
| Table of contents.....             | 1 |
| 1 Introduction.....                | 2 |
| 1.1 Objective.....                 | 2 |
| 1.2 Scope.....                     | 2 |
| 1.3 Glossary.....                  | 2 |
| 1.4 References.....                | 3 |
| 2 Overall Description.....         | 4 |
| 2.1 Survey.....                    | 4 |
| 2.2 Overall requirements.....      | 4 |
| 2.3 Business process.....          | 4 |
| 3 Detailed Requirements.....       | 5 |
| 3.1 Use case 1.....                | 6 |
| 3.2 Use case 2.....                | 7 |
| 4 Supplementary specification..... | 8 |
| 4.1 Functionality.....             | 8 |
| 4.2 Usability.....                 | 8 |
| 4.3 Reliability.....               | 8 |
| 4.4 Performance.....               | 8 |
| 4.5 Supportability.....            | 8 |
| 4.6 Other requirements.....        | 8 |

# 1 Introduction

## 1.1 Objective

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive architectural overview and detailed description of the functional and non-functional requirements for the AIMS (An Internet Media Store) software system.

This document is intended for the following audiences:

- **Analysts/Designers:** This document provides the Analysts and Designers with a clear understanding of the system requirements to design an appropriate solution.
- **Developers:** This document serves as the primary reference for Developers to implement the system features as specified.
- **Testers:** This document enables Testers to derive test cases and verify that the implemented system satisfies the stated requirements.
- **Project Managers:** This document gives Project Managers an overview of the system scope, features, and constraints to facilitate planning and resource allocation.

## 1.2 Scope

AIMS (An Internet Media Store) is an e-commerce software system (web or desktop application) designed for the buying and selling of physical media products, including books, newspapers, CDs, and DVDs. Other types of products may be supported in the future.

The software will:

- Allow customers to browse, search, and filter products by title, category, and price range without requiring authentication.
- Enable customers to add products to a shopping cart, place orders, enter delivery information, and complete payments via VietQR (QR code) or credit card through the PayPal Sandbox gateway.
- Allow product managers to add, view, edit, and delete products, adjust stock quantities, view product operation history, and review pending orders (approve or reject).
- Allow administrators to manage user accounts (create, view, deactivate, block/unblock) and assign or modify user roles.

- Provide authentication (login and change password) for administrators and product managers.

The software will not:

- Support digital media products (e.g., e-books, streaming content).
- Support payment gateways other than PayPal Sandbox and VietQR in the current version.

Key objectives and goals include operating 24/7, serving up to 1,000 concurrent customers without significantly reducing performance, operating continuously for 300 hours without failure, resuming normal operation within a maximum of 1 hour after an incident, and achieving a maximum response time of 2 seconds under normal conditions or 5 seconds during peak hours.

### 1.3 Glossary

| <i>No</i> | <i>Term</i>     | <i>Explanation</i>                                                                          | <i>Example</i>  | <i>Note</i>                         |
|-----------|-----------------|---------------------------------------------------------------------------------------------|-----------------|-------------------------------------|
| 1         | AIMS            | An Internet Media Store, the e-commerce software system described in this document          | AIMS website    | Main system                         |
| 2         | Customer        | External user who browses products, adds products to cart, places orders, and makes payment | Buyer of a book | No login required for shopping flow |
| 3         | Product Manager | Internal user responsible for managing products and reviewing pending orders                | Store staff     | Login required                      |

| <i>No</i> | <i>Term</i>     | <i>Explanation</i>                                                                | <i>Example</i>        | <i>Note</i>                        |
|-----------|-----------------|-----------------------------------------------------------------------------------|-----------------------|------------------------------------|
| 4         | Cart            | Temporary collection of products selected by the customer before placing an order | Shopping cart         | Stores quantity and selected items |
| 5         | Order           | A confirmed request by a customer to purchase one or more products                | Order #AIMS1234       | Created after successful payment   |
| 6         | Invoice         | Payment-related summary of ordered products, VAT, shipping fee, and total amount  | Invoice for one order | Generated during ordering          |
| 7         | VietQR          | External QR-code payment service used for payment integration                     | VietQR sandbox        | Used in UC001                      |
| 8         | Payment Gateway | External service that validates and processes card payment transactions           | PayPal Sandbox        | Used in UC006                      |
| 9         | VAT             | Value-added tax applied to product prices                                         | 10% VAT               | Shipping fee is not subject to VAT |

| <i>No</i> | <i>Term</i>        | <i>Explanation</i>                                                                        | <i>Example</i>     | <i>Note</i>                  |
|-----------|--------------------|-------------------------------------------------------------------------------------------|--------------------|------------------------------|
| 10        | Access Token       | Authentication token returned by an external API and used in later requests               | Bearer token       | Used in VietQR flow          |
| 11        | Callback           | A request sent from an external payment service back to AIMS to notify transaction result | payment callback   | Used by VietQR               |
| 12        | Pending Processing | Order state after successful payment but before approval/rejection by Product Manager     | pending processing | Defined by problem statement |

## 1.4 References

1. AIMS – Draft 1.
2. AIMS – Problem Statement ver3.1.
3. Lab&Assignment02 – Use Case Specification.
4. Lab&Assignment03 – Software Requirement Specification.
5. [TEMPLATE-EN] SRS v1.2.
6. VietQR API documentation for token, QR generation, and callback testing.
7. PayPal Sandbox / payment API documentation for card payment and refund.

## 2 Overall Description

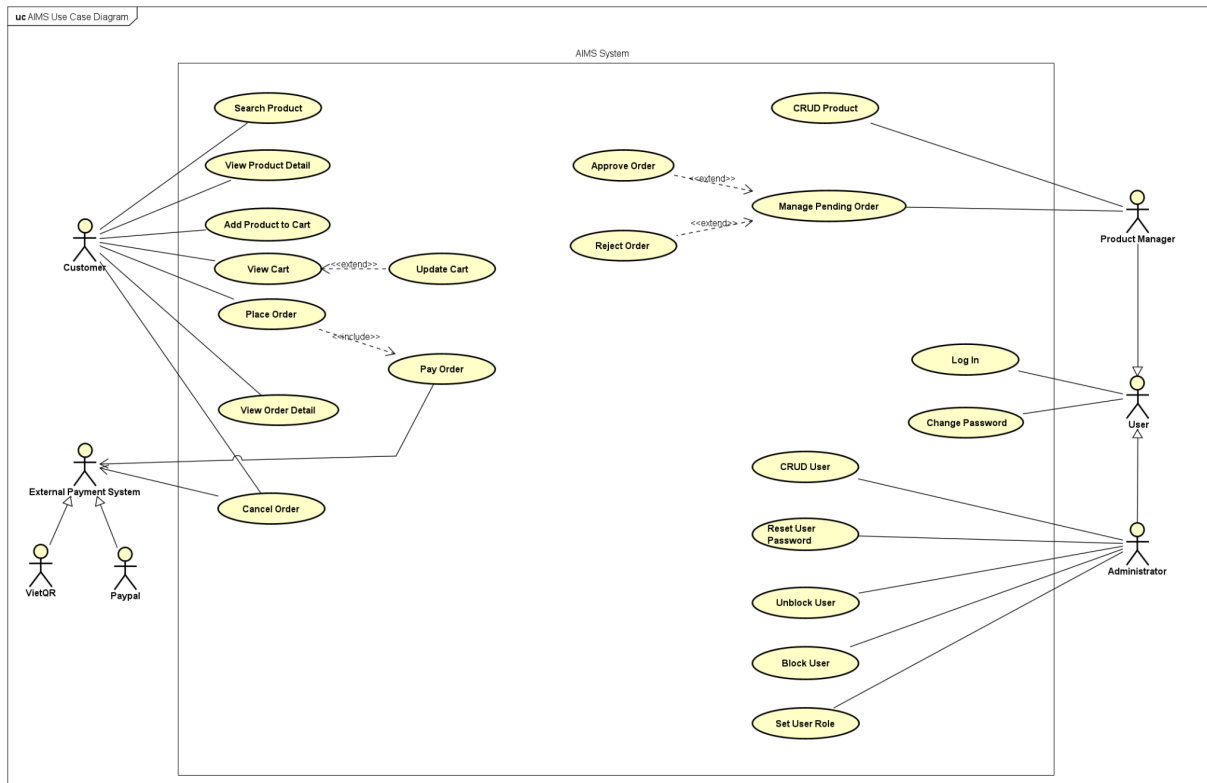
### 2.1 Survey

AIMS is an online media store that supports the sale of physical media products such as books, newspapers, CDs, and DVDs. The system is designed to operate continuously and serve customers in a convenient and stable manner. Customers can browse available products, search by title or category, filter by price range, view detailed information of products, add products to cart, place orders, and pay electronically. Product Managers can manage products and review or reject pending orders. Administrators manage user accounts and permissions

**Main actors of the system are:**

1. **Customer:** browses/searches products, views product details, adds products to cart, places orders, and pays for orders.
2. **Product Manager:** creates, updates, deletes, and views products; reviews pending orders.
3. **Administrator:** manages users, roles, blocking/unblocking, password reset, and user information.
4. **VietQR / Payment Gateway:** external systems used to process payment transactions.

## 2.2 Overall requirements



## 2.3 Business process

### 2.3.1 Customer Purchasing Process

This process describes the end-to-end flow of a customer browsing products and completing a purchase:

1. The customer opens the application. The system displays a list of 20 random products.
2. The customer searches for products by title or category, or filters by price range. The system displays matching results.
3. The customer selects a product to view its detailed information (including type-specific attributes).
4. The customer adds products with corresponding quantities to the shopping cart.



5. The customer views the cart. The system displays the total price (excluding VAT), the list of products with quantities and prices, and any stock insufficiency warnings.
6. The customer proceeds to place an order. The system checks stock availability. If insufficient, the customer is prompted to update the cart.
7. The customer enters delivery information (name, phone, province, address). The system validates the input and calculates the shipping fee.
8. The system displays the invoice: product list, quantities, prices, total excluding VAT, total including VAT, delivery fee, and total amount to be paid.
9. The customer selects a payment method. By default, VietQR (QR code) is displayed. The customer may switch to credit card payment via PayPal.
10. Upon successful payment, the system displays order and transaction information, sets the order to pending processing state, and sends an invoice and transaction email to the customer.
11. The customer may later view order information or cancel the order (before approval) using links provided in the email.

### 2.3.2 Product Management Process

This process describes how product managers maintain the product catalog:

1. The product manager logs into the system.
2. The product manager selects a product management action: add, edit, or delete a product.
3. For adding: the product manager enters common product information and type-specific details. The system validates the input (including the 30%–150% price constraint) and saves the product.
4. For editing: the product manager selects a product, modifies its information (including manual stock adjustments with recorded reasons), and the system validates and updates the record.

5. For deleting: the product manager selects up to 10 products. The system checks the daily deletion limit (20/day) and each product's stock. Products with stock = 0 are permanently deleted; products with stock > 0 are deactivated.
6. The system records all operations in the product history log. The product manager can query this history at any time.
7. The system notifies the product manager if any operation is invalid (e.g., non-compliant input, incorrect date format).

### 2.3.3 Order Fulfillment Process

This process describes how product managers review and process customer orders:

1. The product manager logs into the system and navigates to the order review interface.
2. The system displays 30 pending orders per page.
3. The product manager selects an order to view its details (invoice, delivery information, payment transaction).
4. The product manager approves or rejects the order. Rejection may occur for various reasons (e.g., undeliverable items, out of stock while the customer was paying, item not found in stock).
5. The system updates the order status and sends a notification email to the customer.
6. If the order is rejected, the system initiates a refund: for PayPal payments, an automatic refund is processed; for VietQR payments, the system notifies the product manager to issue a manual refund.

### 2.3.4 User Management Process

This process describes how administrators manage user accounts and roles:

1. The administrator logs into the system.

2. The administrator selects a user management action: create, view, deactivate, block, or unblock a user account.
3. For creating: the administrator provides user information and assigns initial roles. The system creates the account.
4. For deactivating/blocking/unblocking: the administrator selects a user and performs the action. The system logs the action and notifies the affected user.
5. The administrator may assign or modify user roles (administrator, product manager). Each user can have multiple roles.
6. For sensitive actions (email updates, password resets), the system automatically logs the action and sends a notification to the affected user. The administrator cannot access users' actual passwords.

## 3 Detailed Requirements

### 3.1 Use case 1

#### Use Case Pay Order (Customer/VietQR)

##### 1. Use case code

UC001

##### 2. Brief Description

This use case describes the interaction between the Customer, AIMS software, and the VietQR payment gateway when the Customer wishes to pay for an order using a VietQR QR code.

##### 3. Actors

**3.1 Customer:** The person who wants to pay for an order.

**3.2 VietQR:** The external payment gateway system that provides QR code generation and payment callback services.

##### 4. Preconditions

- The use case "Place Order" has been initiated, and the invoice information has been calculated and temporarily saved.
- The customer proceeds to pay for the order by QR code.

##### 5. Basic Flow of Events

1. AIMS software calls the API of VietQR to get the access token.
2. VietQR validates the credentials and returns the access token to AIMS software.
3. AIMS software calls the API of VietQR to generate the QR code.
4. VietQR generates and returns the QR code data to AIMS software.
5. AIMS software displays the QR code on the payment screen together with the invoice information.
6. Customer triggers VietQR's test callback API to simulate a successful payment transaction.
7. VietQR sends a payment callback to AIMS software with the transaction result.
8. AIMS software verifies the callback data.
9. AIMS software records the payment transaction information.
10. AIMS software returns the payment result to the use case "Place Order".

##### 6. Alternative flows

Table Alternative flows of events for UC Place order

| No | Location  | Condition                                            | Action                                                                                                                                           | Resume location |
|----|-----------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 1. | At Step 2 | If VietQR fails to return a valid access token       | AIMS software notifies the customer that the payment service is temporarily unavailable and suggests retrying or choosing another payment method | Use case ends   |
| 2. | At Step 4 | If VietQR fails to generate the QR code              | AIMS software notifies the customer that the payment service is temporarily unavailable and suggests retrying or choosing another payment method | Use case ends   |
| 3. | At Step 6 | If the customer cancels the payment                  | AIMS software cancels the current payment session                                                                                                | Use case ends   |
| 4. | At Step 7 | If the callback indicates the transaction has failed | AIMS software notifies the customer that the payment has failed                                                                                  | Use case ends   |

## 7. Input data

Table 7.1 - Input data for Get Token API

| No | Data fields | Description                                                   | Mandatory | Valid condition                     | Example                  |
|----|-------------|---------------------------------------------------------------|-----------|-------------------------------------|--------------------------|
| 1. | username    | Used for Basic Authentication when requesting an access token | Yes       | Non-empty string provided by VietQR | customer-vietqrtest-user |
| 2. | password    | Used for Basic Authentication when requesting an access token | Yes       | Non-empty string provided by VietQR | Y3VzdG9t...              |

Table 7.2 - Input data for API Generate QR Code

| No | Data fields  | Description                                                                                                      | Mandatory | Valid condition                                                 | Example                  |
|----|--------------|------------------------------------------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------|--------------------------|
| 1. | bankCode     | Bank code of the account.                                                                                        | Yes       | Valid bank code                                                 | MB                       |
| 2. | bankAccount  | Bank account generates VietQR payment code.                                                                      | Yes       | Non-empty string                                                | 0333673636               |
| 3. | userBankName | Account holder's full name                                                                                       | Yes       | Latin/unaccented characters only                                | TO LAM                   |
| 4. | content      | Payment description                                                                                              | Yes       | Max 19 characters; Latin/unaccented only; no special characters | TAP CAN BINH chuyen tien |
| 5. | qrType       | Dynamic VietQR                                                                                                   | Yes       | Integer value: 0                                                | 0                        |
| 6. | amount       | Total payment amount in VND                                                                                      | Yes       | Positive integer equals total amount of the invoice             | 36000                    |
| 7. | orderId      | Transaction ID code needs to be managed.<br>“orderId” will be returned when the system receives a balance change | Yes       |                                                                 | AIMS1234                 |
| 8. | serviceCode  | Code of products and services to be paid.                                                                        | No        |                                                                 | BOOKID123                |

Table 7.3 - Input data for API Test Callback

| No | Data fields | Description                                 | Mandatory | Valid condition                                    | Example                  |
|----|-------------|---------------------------------------------|-----------|----------------------------------------------------|--------------------------|
| 1. | bankAccount | Bank account generates VietQR payment code. | Yes       | Must match the bankAccount used in Generate QR API | 0333673636               |
| 2. | content     | Money transfer content.                     | Yes       | Must exactly match Generate QR response "content"  | TAP CAN BINH chuyen tien |

## 8. Output data

Table 8.1 - Output data from API Get Token

| No | Data fields  | Description                                                                                  | Display format | Example     |
|----|--------------|----------------------------------------------------------------------------------------------|----------------|-------------|
| 1. | access_token | It is the Bearer Token provided by VietQR. Used to access and generate VietQR payment codes. | String         | eyJhbGci... |
| 2. | token_type   | It is a "Bearer" type token.                                                                 | String         | Bearer      |

Table 8.2 - Output data from API Generate QR Code

| No | Data fields | Description                                                                              | Display format | Example                                      |
|----|-------------|------------------------------------------------------------------------------------------|----------------|----------------------------------------------|
| 1. | qrCode      | QR code data encoded as a string. Used to render the QR image in the AIMS payment screen | String         | 000201010212...                              |
| 2. | qrLink      | URL linking to the hosted QR code image at VietQR. Can be embedded in an <img> tag       | URL String     | https://pro.vietqr.vn/qr-generated?token=... |

|    |                  |                                                       |        |             |
|----|------------------|-------------------------------------------------------|--------|-------------|
| 3. | transactionRefId | VietQR's unique reference ID for this QR transaction. | String | MGEzMDIz... |
|----|------------------|-------------------------------------------------------|--------|-------------|

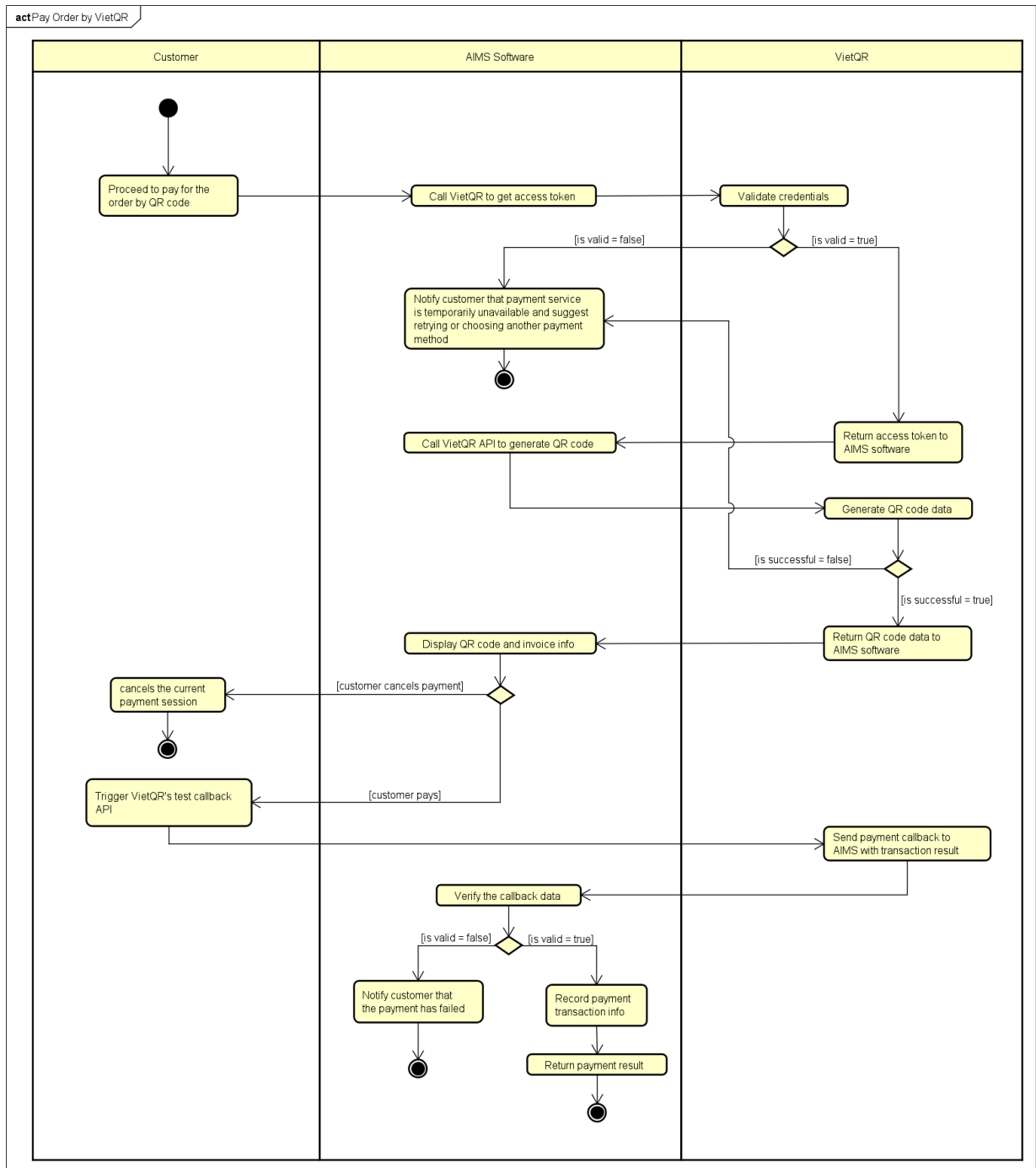
Table 8.3 - Output data from API Test Callback

| No | Data fields | Description                                              | Display format        | Example           |
|----|-------------|----------------------------------------------------------|-----------------------|-------------------|
| 1. | status      | Result status of the test callback simulation.           | "SUCCESS" or "FAILED" | SUCCESS           |
| 2. | message     | The transaction ID; or error code and error description. | String                | ID_of_transaction |

## 9. Postconditions

The payment result is returned to use case "Place Order". Use case "Place Order" then records the order, and notifies the Customer or nothing happens if payment is not successful.





Activity Diagram - Pay Order (Customer/VietQR)

## 3.2 Use case 2

### Use Case Create Product

#### 1. Use case code

UC002

#### 2. Brief Description

This use case describes how the Product Manager creates a new product in the AIMS system.

#### 3. Actors

**3.1 Product Manager:** The internal user responsible for creating product in the system.

#### 4. Preconditions

- Product Manager has logged into the system
- Product Manager has permission to manage products

#### 5. Basic Flow of Events

1. Product Manager opens the Product Management page
2. Product Manager selects “Create Product”
3. System displays product creation form
4. Product Manager enters product information
5. Product Manager submits the form
6. System validates input data
7. System saves the new product into database
8. System displays success message

#### 6. Alternative flows

**Table Alternative flows of events for UC Create Product**

| No | Location  | Condition             | Action                                          | Resume location |
|----|-----------|-----------------------|-------------------------------------------------|-----------------|
| 1  | At Step 6 | Input data is invalid | System shows error message and asks to re-enter | Step 4          |

#### 7. Input data

**Table 7.1 - Input data for Create Product**

| No | Data fields | Description | Mandatory | Valid condition | Example |
|----|-------------|-------------|-----------|-----------------|---------|
|----|-------------|-------------|-----------|-----------------|---------|

|   |             |                            |     |                      |                       |
|---|-------------|----------------------------|-----|----------------------|-----------------------|
| 1 | productName | Name of the product        | Yes | Non-empty string     | Java Programming Book |
| 2 | productType | Category of product        | Yes | Valid product type   | Book                  |
| 3 | price       | Price of the product       | Yes | Positive number      | 120000                |
| 4 | quantity    | Available stock quantity   | Yes | Non-negative integer | 50                    |
| 5 | description | Description of the product | No  | Text                 | Programming textbook  |

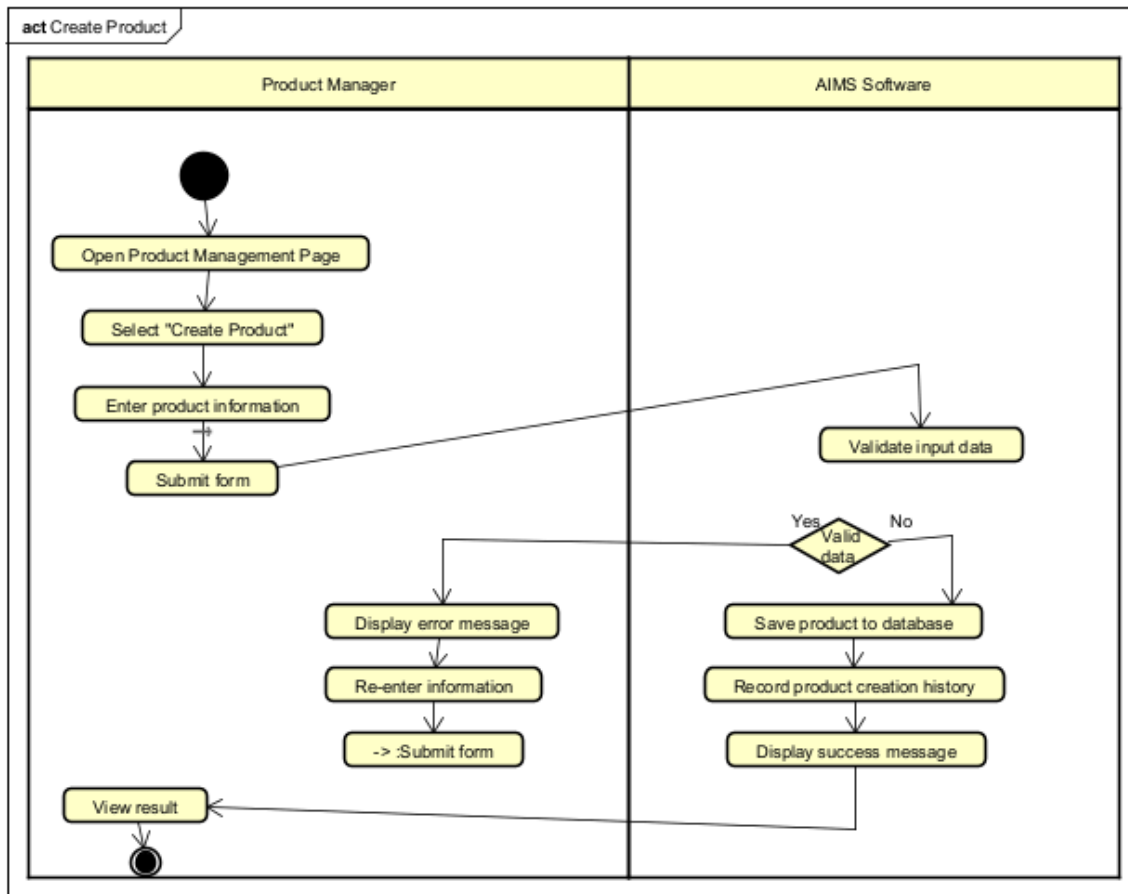
## 8. Output data

**Table 8.1 - Output data for Create Product**

| No | Data fields | Description                     | Display format | Example                      |
|----|-------------|---------------------------------|----------------|------------------------------|
| 1  | status      | Result of the create operation  | String         | SUCCESS                      |
| 2  | message     | Notification message            | String         | Product created successfully |
| 3  | productId   | ID of the newly created product | String         | P102                         |

## 9. Postconditions

1. New product is stored in database
2. Product appears in product list



## Use Case Update Product

### 1. Use case code

UC003

### 2. Brief Description

This use case describes how the Product Manager updates existing product information.

### 3. Actors

**3.1 Product Manager:** The internal user responsible for updating product information in the system.

### 4. Preconditions

- Product Manager is logged in
- Product exists in database

### 5. Basic Flow of Events

1. Product Manager opens Product Management page
2. System displays product list

3. Product Manager selects a product
4. System displays product information
5. Product Manager edits product information
6. Product Manager submits changes
7. System validates input
8. System updates product in database
9. System displays success message

## 6. Alternative flows

**Table Alternative flows of events for UC Update Product**

| No | Location  | Condition             | Action                                     | Resume location |
|----|-----------|-----------------------|--------------------------------------------|-----------------|
| 1  | At Step 7 | Input data is invalid | System shows error and requests correction | Step 5          |
| 2  | At Step 3 | Product not found     | System shows error message                 | Use case ends   |

## 7. Input data

**Table 7.1 - Input data for Update Product**

| No | Data fields | Description                      | Mandatory | Valid condition        | Example            |
|----|-------------|----------------------------------|-----------|------------------------|--------------------|
| 1  | productId   | Unique identifier of the product | Yes       | Must exist in database | P001               |
| 2  | productName | Updated product name             | No        | Non-empty string       | Advanced Java Book |
| 3  | price       | Updated price                    | No        | Positive number        | 150000             |
| 4  | quantity    | Updated stock quantity           | No        | Non-negative integer   | 30                 |

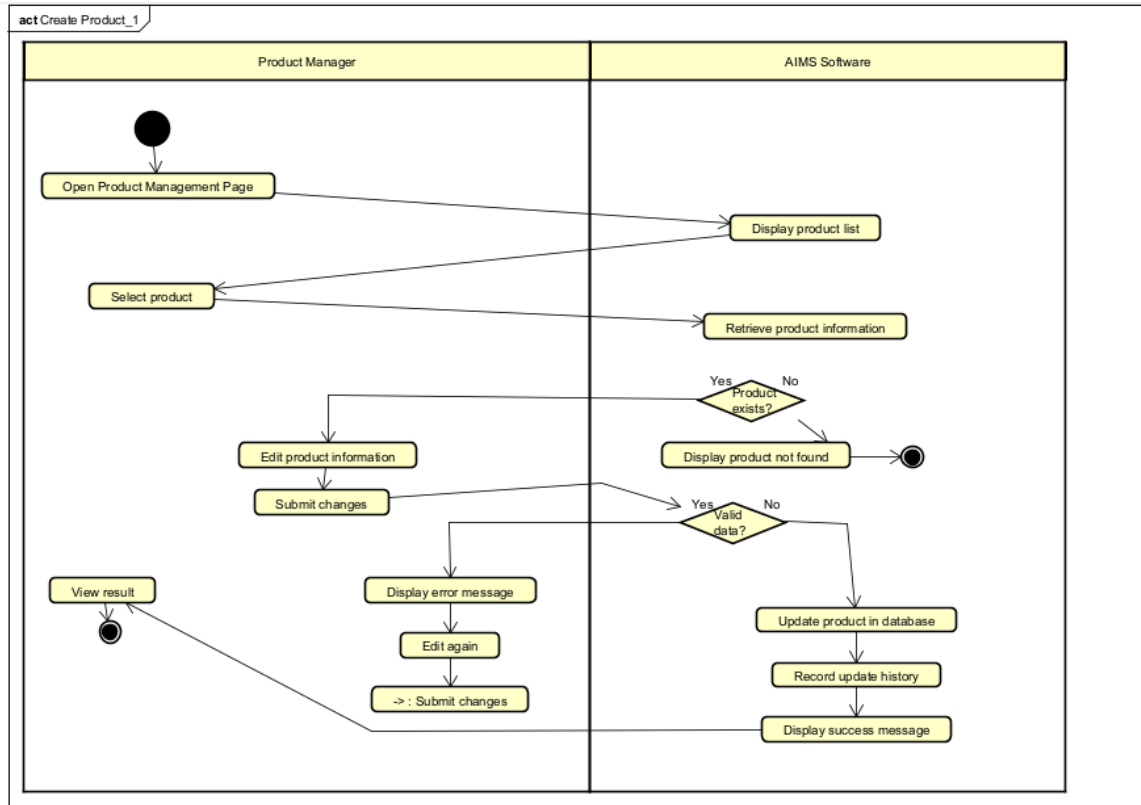
## 8. Output data

**Table 8.1 - Output data for Update Product**

| No | Data fields | Description                    | Display format | Example                      |
|----|-------------|--------------------------------|----------------|------------------------------|
| 1  | status      | Result of the update operation | String         | SUCCESS                      |
| 2  | message     | Notification message           | String         | Product successfully updated |
| 3  | productId   | ID of the updated product      | String         | P102                         |

## 9. Postconditions

1. Product information is updated in database



## Use Case Delete Product (Product Manager)

### 1. Use case code

UC004

### 2. Brief Description

This use case describes how the Product Manager deletes a product from the system.

### 3. Actors

**3.1 Product Manager:** The internal user responsible for deleting product in the system.

### 4. Preconditions

- Product Manager is logged in
- Product exists in database

### 5. Basic Flow of Events

1. Product Manager opens Product Management page
2. System displays product list
3. Product Manager selects a product
4. Product Manager clicks "Delete"

5. System asks for confirmation
6. Product Manager confirms deletion
7. System removes product from database
8. System displays success message

## 6. Alternative flows

**Table Alternative flows of events for UC Delete Product**

| No | Location  | Condition             | Action                                        | Resume location |
|----|-----------|-----------------------|-----------------------------------------------|-----------------|
| 1  | At Step 6 | User cancels          | System cancels operation                      | Use case ends   |
| 2  | At Step 3 | Product not found     | System shows error                            | Use case ends   |
| 3  | At Step 7 | Product has stock > 0 | System does not delete, mark as “deactivated” | Use case ends   |

## 7. Input data

**Table 7.1 - Input data for Delete Product**

| No | Data fields | Description                                | Mandatory | Valid condition        | Example |
|----|-------------|--------------------------------------------|-----------|------------------------|---------|
| 1  | productId   | Unique identifier of the product to delete | Yes       | Must exist in database | P001    |

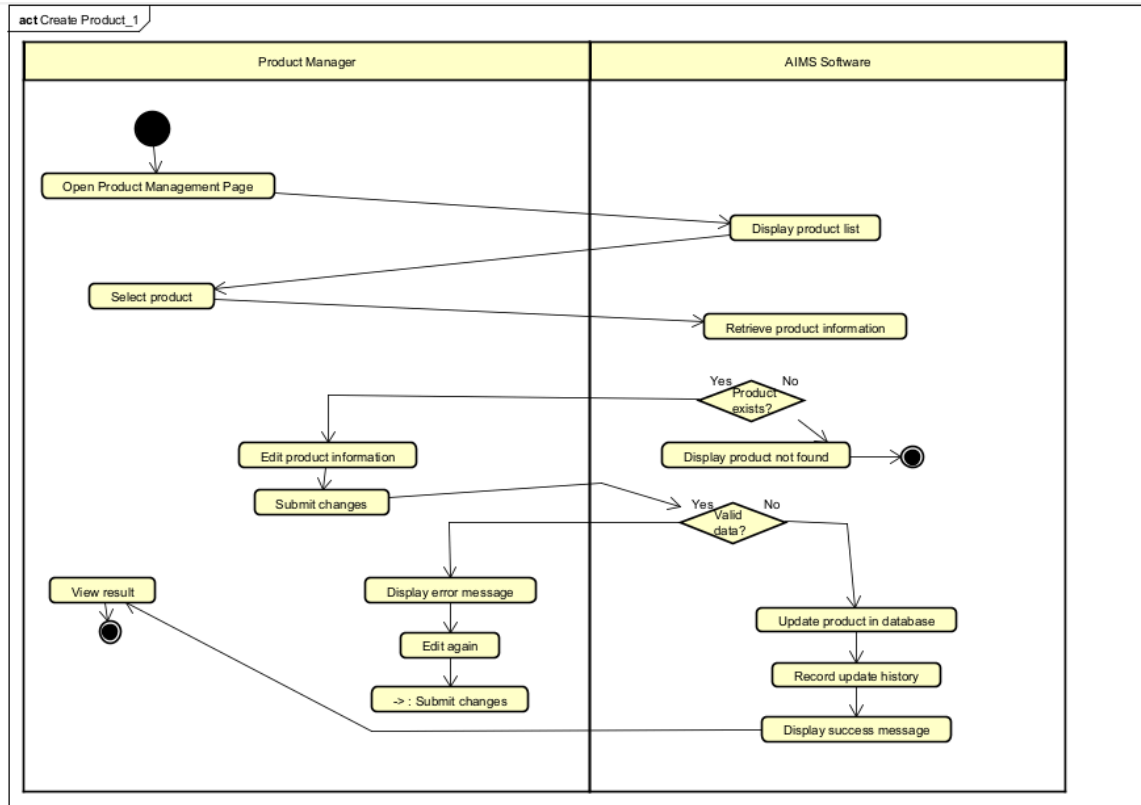
## 8. Output data

**Table 8.1 - Output data for Delete Product**

| No | Data fields | Description                    | Display format | Example                      |
|----|-------------|--------------------------------|----------------|------------------------------|
| 1  | status      | Result of the delete operation | String         | SUCCESS                      |
| 2  | message     | Notification message           | String         | Product successfully deleted |
| 3  | productId   | ID of the deleted product      | String         | P102                         |

## 9. Postconditions

Product is deleted OR marked as deactivated



### 3.3 Use case 3

## Use Case View Product Detail

### 1. Use case code

UC003

### 2. Brief Description

This use case describes how a Customer or Product Manager selects a product from a product list/search result and views its detailed information, including general attributes and type-specific attributes depending on the product type.

### 3. Actors

**3.1 Customer:** The external user who browses products and views detailed information before deciding whether to purchase.

**3.2 Product Manager:** The internal user who manages products and may view detailed information of a product in the system.

**3.3 AIMS Software:** The system that retrieves and displays product information.



#### 4. Preconditions

- The AIMS system is running.
- The actor is accessing a screen that displays a product list or search results.
- If the actor is a Product Manager, the actor has logged in with the proper role.
- The actor has selected a product item to view.

#### 5. Basic Flow of Events

1. The actor selects a product from the product list or search result.
2. The AIMS software receives the request to view product details.
3. The AIMS software retrieves the general information of the selected product.
4. The AIMS software determines the product type and retrieves the corresponding type-specific information.
5. The AIMS software displays the detailed product information.
6. The actor views the product details.

#### 6. Alternative flows

**Table 6.1-Alternative flows of events for UC View Product Detail**

| No | Location       | Condition                                                                                        | Action                                               | Resume location |
|----|----------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------|-----------------|
| 1  | At Step 3      | if the selected product no longer exists or cannot be found                                      | AIMS software displays a “Product not found” message | Use case ends   |
| 2  | At Step 3 or 4 | If the system cannot retrieve general or type-specific product information due to a system error | AIMS software displays an error message              | Use case ends   |

#### 7. Input data

**Table 7.1 – Input data for View Product Detail**

| No | Data fields | Description | Mandatory | Valid condition | Example |
|----|-------------|-------------|-----------|-----------------|---------|
|----|-------------|-------------|-----------|-----------------|---------|

|   |           |                                           |     |                        |      |
|---|-----------|-------------------------------------------|-----|------------------------|------|
| 1 | productId | Unique identifier of the selected product | Yes | Must exist in database | P001 |
|---|-----------|-------------------------------------------|-----|------------------------|------|

## 8. Output data

**Table 8.1 – Output data for View Product Detail**

| No | Data fields   | Description                          | Display format  | Example               |
|----|---------------|--------------------------------------|-----------------|-----------------------|
| 1  | barcode       | Barcode of product                   | String          | 8931234567890         |
| 2  | title         | Title of product                     | String          | Java Programming Book |
| 3  | category      | Product category                     | String          | Book                  |
| 4  | originalValue | Original value of product            | Positive number | 150000                |
| 5  | currentPrice  | Current selling price                | Positive number | 120000                |
| 6  | description   | General description of product       | Text            | New book              |
| 7  | dimensions    | Height, width, and length of product | Number(s)       | 24 x 16 x 3 cm        |
| 8  | weight        | Weight of product                    | Positive number | 0.5 kg                |

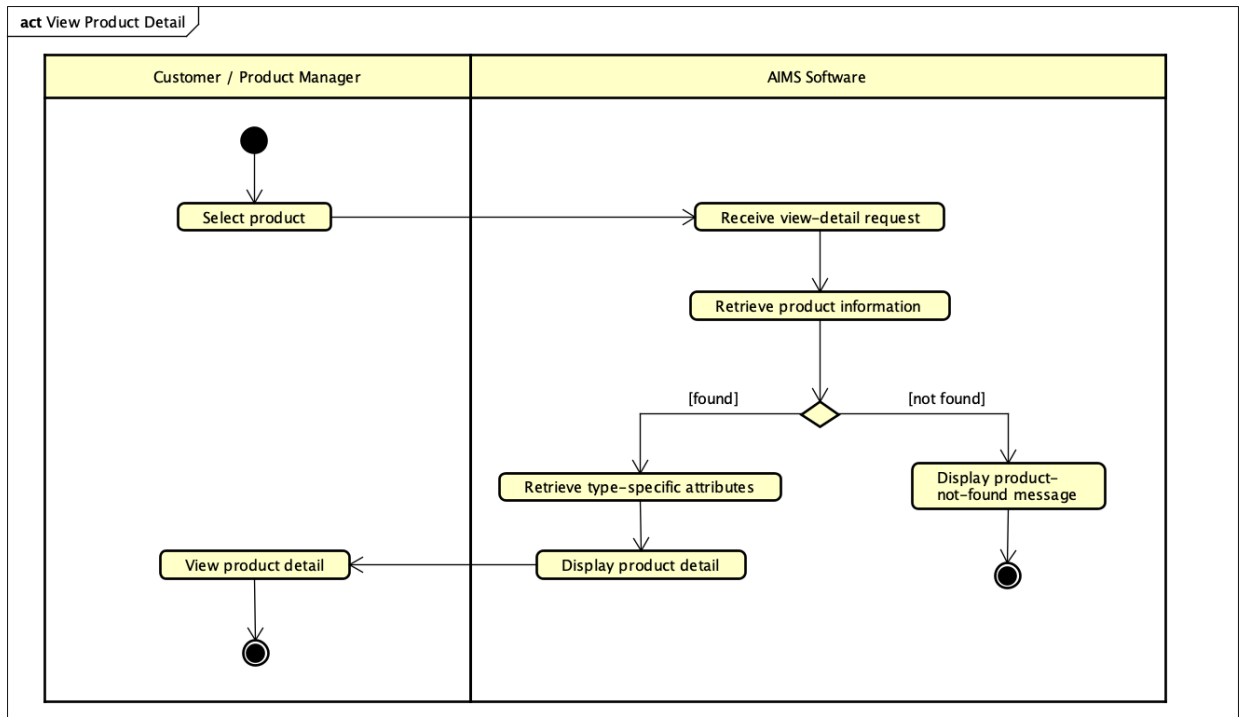
**Note:** originalValue and currentPrice are displayed excluding 10% VAT.

**Table 8.2 – Type-specific output data**

| Product type | Additional fields                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------|
| Book         | author(s), cover type, publisher, publication date, number of pages, language, genre                        |
| Newspaper    | editor-in-chief, publisher, publication date, issue number, publication frequency, ISSN, language, sections |
| CD           | artists, record labels, tracks list, genre, release date                                                    |
| DVD          | disc type, director, runtime, studio, language, subtitles, release date, genre                              |

## **9. Postconditions**

- The detailed information of the selected product is displayed to the actor.
- No product data is modified.



### 3.4 Use Case 4

## Use Case Add Product to Cart

### 1. Use case code

UC004

### 2. Brief Description

This use case describes the interaction between the Customer and the AIMS software when the Customer wishes to add products to their shopping cart.

### 3. Actors

**3.1. Customer:** The person who wants to add a product to the cart.

### 4. Preconditions

The Customer is viewing the products list screen or product detail screen. No login required.

### 5. Basic Flow of Events

1. The Customer selects a product and enters the desired quantity.
2. The Customer requests to add the products to the cart.
3. The software adds the product and its quantity to the current cart.

### 6. Alternative flows

**Table 1-Alternative flows of events for UC Add Product to Cart**

| No | Location | Condition                         | Action                                                                               | Resume location |
|----|----------|-----------------------------------|--------------------------------------------------------------------------------------|-----------------|
| 1. | Any step | If the Customer exit the software | <ul style="list-style-type: none"><li>• The software saves the cart state.</li></ul> | Use Case ends.  |

### 7. Input data

**Table A-Input data of Add Product to Cart**

| No | Data fields | Description                                              | Mandatory | Valid condition                       | Example |
|----|-------------|----------------------------------------------------------|-----------|---------------------------------------|---------|
| 1. | Product ID  | The unique identifier of the selected product.           | Yes       | Must correspond to an active product. | BO0001  |
| 2. | Quantity    | The number of said product that the Customer want to add | Yes       | Must be an integer bigger than 0.     | 1       |

## 8. Output data

**Table B-Output data of Add Product to Cart**

| No | Data fields          | Description                                           | Display format | Example                       |
|----|----------------------|-------------------------------------------------------|----------------|-------------------------------|
|    | Success Notification | A message indicating the item was added successfully. | Text           | “Added to cart successfully.” |

## 9. Postconditions

The selected product and its specified quantity are successfully added to the Customer's current shopping cart.

### 3.5 Use Case 5

## Use Case Place Order

### 1. Use case code

UC005

### 2. Brief Description

This use case describes the interaction between the Customer and the AIMS software when the Customer proceeds to place an order, enters the delivery information and reviews the invoice before the payment.

### 3. Actors

**3.1. Customer:** The person who wants to place an order.

### 4. Preconditions

The Customer is viewing the cart and there is at least one product in the cart.

### 5. Basic Flow of Events

1. The Customer requests to place an order from the cart.
2. The software checks if the available quantity of the product(s) is sufficient.
3. The software asks the Customer to provide delivery information.
4. The Customer enters delivery information (see Table A).
5. The software validates the Customer's input.
6. The software calculates the delivery fee.
7. The software displays and temporarily saves the invoice information (see Table B).
8. Customer requests to pay order.
9. The software calls UC "Pay Order" to handle the payment process.
10. The software sends the invoice and payment transaction information to the Customer's email.
11. The software records the payment transaction information and the successfully paid order.
12. The software empties the Customer's cart and displays a successful order screen.

### 6. Alternative flows

**Table N-Alternative flows of events for UC Place order**

| No | Location | Condition                                                                    | Action                                                                                                                                                                     | Resume location      |
|----|----------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1. | Step 2   | If the stock is insufficient for any product.                                | <ul style="list-style-type: none"> <li>The software prompts the Customer to update the cart</li> <li>Displays the available quantity for each affected product.</li> </ul> | Resumes at Step 1    |
| 2. | Step 5   | If any of the required fields is left blank or contains invalid information. | <ul style="list-style-type: none"> <li>Ask the Customer to fill in the form again.</li> </ul>                                                                              | Resumes at Step 4    |
| 3. | Step 6   | If the Customer adjusts the delivery method or items they wish to purchase.  | <ul style="list-style-type: none"> <li>The software recalculates the delivery fees and updates the invoice.</li> </ul>                                                     | Resumes at Step 6    |
| 4. | Step 9   | UC Pay Order returns payment fails                                           | <ul style="list-style-type: none"> <li>The software shows a payment error</li> </ul>                                                                                       | Use case ends.       |
| 5. | Step 10  | Payment successes but email sending fails.                                   | <ul style="list-style-type: none"> <li>The software marks the order as “email_error” in the order note.</li> </ul>                                                         | Continues to Step 11 |
| 6. | Any step | If the Customer exits the software.                                          | <ul style="list-style-type: none"> <li>The software terminates the process.</li> </ul>                                                                                     | Use case ends.       |

## 7. Input data

**Table A-Input data of delivery information**

| No | Data fields   | Description            | Mandatory | Valid condition                           | Example      |
|----|---------------|------------------------|-----------|-------------------------------------------|--------------|
| 1. | Customer name | Name of the recipient. | Yes       | Text characters only, 3 – 100 characters. | Nguyen Van A |



|    |                  |                                                            |     |                                                                                          |                              |
|----|------------------|------------------------------------------------------------|-----|------------------------------------------------------------------------------------------|------------------------------|
| 2. | Contact email    | Email address for contract                                 | Yes | Valid email format: username@example.com                                                 | username@example.com         |
| 3. | Phone number     | Contact number                                             | Yes | Valid phone number format: 9 – 10 digits, start with ‘0’ or ‘+84’ (for Vietnam numbers). | 0193 918 484                 |
| 4. | Shipping Address | Detailed delivery address                                  | Yes | Must not be blank                                                                        | 87 Dai Co Viet, Hai Ba Trung |
| 5. | Province/City    | Delivery province/city, used for calculating delivery fee. | Yes | Valid province/city in Vietnam                                                           | Hanoi                        |
| 6. | Delivery Method  | Chosen method of delivery.                                 | Yes | One of the methods provided by software.                                                 | Fast shipping                |

## 8. Output data

**Table B-Output data of invoice information**

| No | Data fields      | Description            | Display format | Example          |
|----|------------------|------------------------|----------------|------------------|
| 1. | List of products | Products in the cart   | Text list      | Book: Clean code |
| 2. | Quantity         | Number of each product | Integer        | 1                |
| 3. | Product Price    | Price of each item     | Currency (VND) | 120,000 VND      |

|    |                           |                                                |                |             |
|----|---------------------------|------------------------------------------------|----------------|-------------|
| 4. | Total Price (without VAT) | Sum of product prices without tax              | Currency (VND) | 120,000 VND |
| 5. | Total Price (with VAT)    | Sum of product prices with 10% VAT             | Currency (VND) | 132,000 VND |
| 6. | Delivery Fee              | Shipping fee based on weight and province/city | Currency (VND) | 18,000 VND  |
| 7. | Total Amount              | Total to be paid: Total Price + Delivery Fee   | Currency (VND) | 150,000 VND |

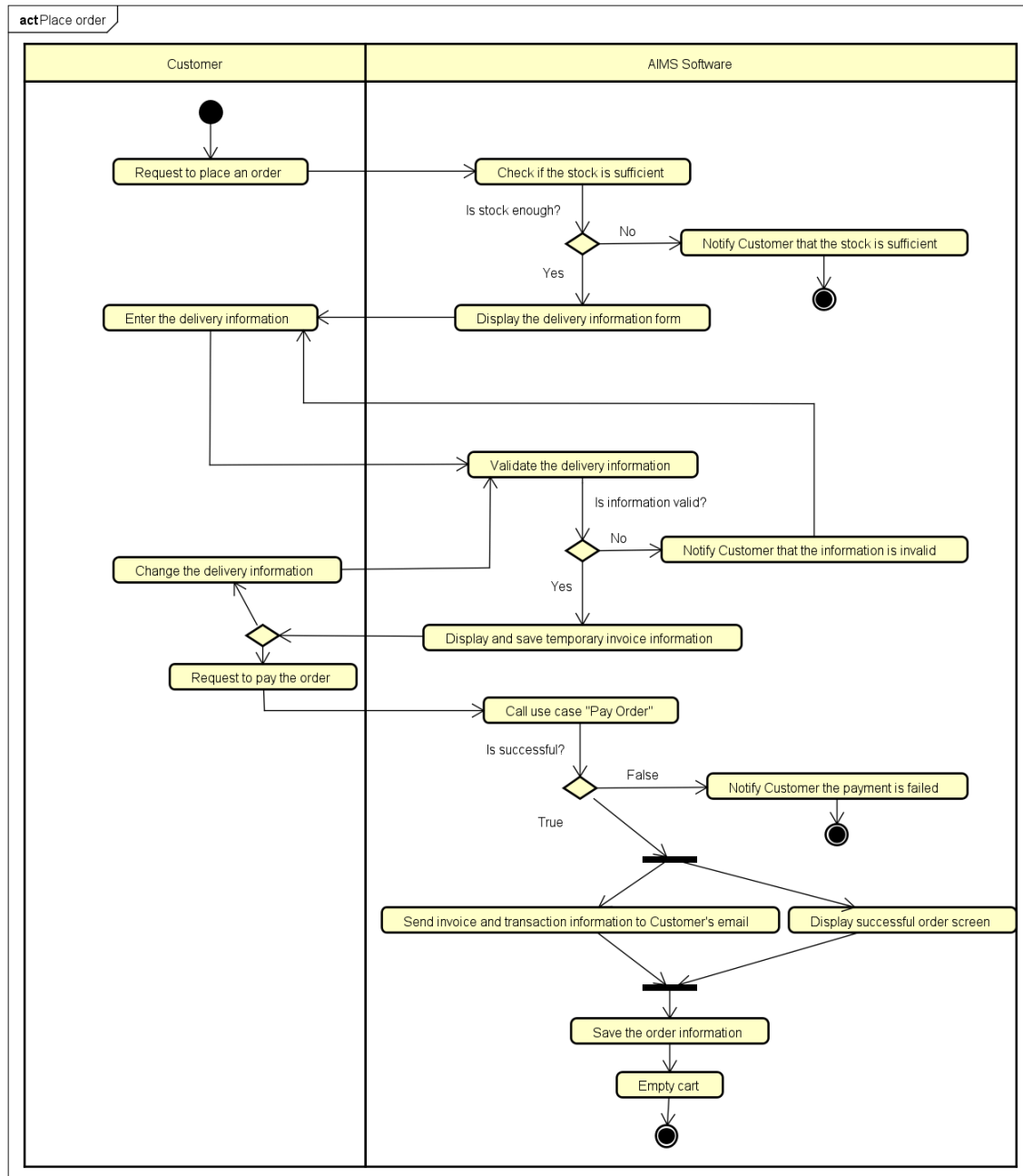
## 9. Postconditions

Successful Conditions:

- Order created with status “Ordered”.
- Inventory updated to reserve ordered products.
- Payment transaction recorded.

Failure Conditions:

- Order not created if any step fails.
- Inventory not updated if payment is not completed.



### 3.6 Use case 6

## Use Case Pay Order (Customer / Credit Card)

### 1. Use case code

UC006

## **2. Brief Description**

This use case describes the interaction between the Customer, the AIMS software, and the Credit Card payment gateway when the customer wants to pay for an order using a credit card.

## **3. Actors**

### **3.1 Customer**

The person who wants to pay for an order.

### **3.2 Payment Gateway**

The external payment service responsible for verifying credit card information and processing the payment transaction.

## **4. Preconditions**

4.1 The use case Place Order has been initiated and the invoice information has been calculated.

4.2 The customer chooses Credit Card as the payment method.

## **5. Basic Flow of Events**

1. AIMS software displays the credit card payment form.
2. The customer enters credit card information.
3. The customer confirms the payment.
4. AIMS software sends the payment request to the payment gateway.
5. The payment gateway validates the credit card information and processes the transaction.

6. The payment gateway returns the transaction result to AIMS software.
7. AIMS software verifies the transaction result.
8. AIMS software records the payment transaction information.
9. AIMS software displays the payment result to the customer.
10. AIMS software returns the payment result to the Place Order use case.

## 6. Alternative flows

| No | Location  | Condition                       | Action                                                                                                                      | Resume Location |
|----|-----------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------|
| 1  | At Step 5 | Invalid credit card information | AIMS software notifies the customer that the card information is invalid and asks the customer to re-enter the card details | Step 2          |
| 2  | At Step 6 | Payment transaction failed      | AIMS software notifies the customer that the payment has failed                                                             | Use case ends   |
| 3  | At Step 2 | Customer cancels the payment    | AIMS software cancels the payment process                                                                                   | Use case ends   |

## 7. Input data

| No | Data fields    | Description                         | Mandatory | Valid condition                         | Example          |
|----|----------------|-------------------------------------|-----------|-----------------------------------------|------------------|
| 1  | cardNumber     | Credit card number used for payment | Yes       | 16-digit numeric value                  | 4111111111111111 |
| 2  | cardHolderName | Name of the card holder             | Yes       | Alphabet characters                     | NGUYEN VAN A     |
| 3  | expirationDate | Expiration date of the credit card  | Yes       | Valid future date                       | 12/27            |
| 4  | cvv            | Security code of the credit card    | Yes       | 3–4 digit numeric value                 | 123              |
| 5  | amount         | Total payment amount                | Yes       | Positive integer equal to invoice total | 36000            |
| 6  | orderId        | Unique ID of the order transaction  | No        | Valid order ID generated by system      | AIMS1234         |

### 8. Output data

| No | Data fields | Description | Display format | Example |
|----|-------------|-------------|----------------|---------|
|----|-------------|-------------|----------------|---------|

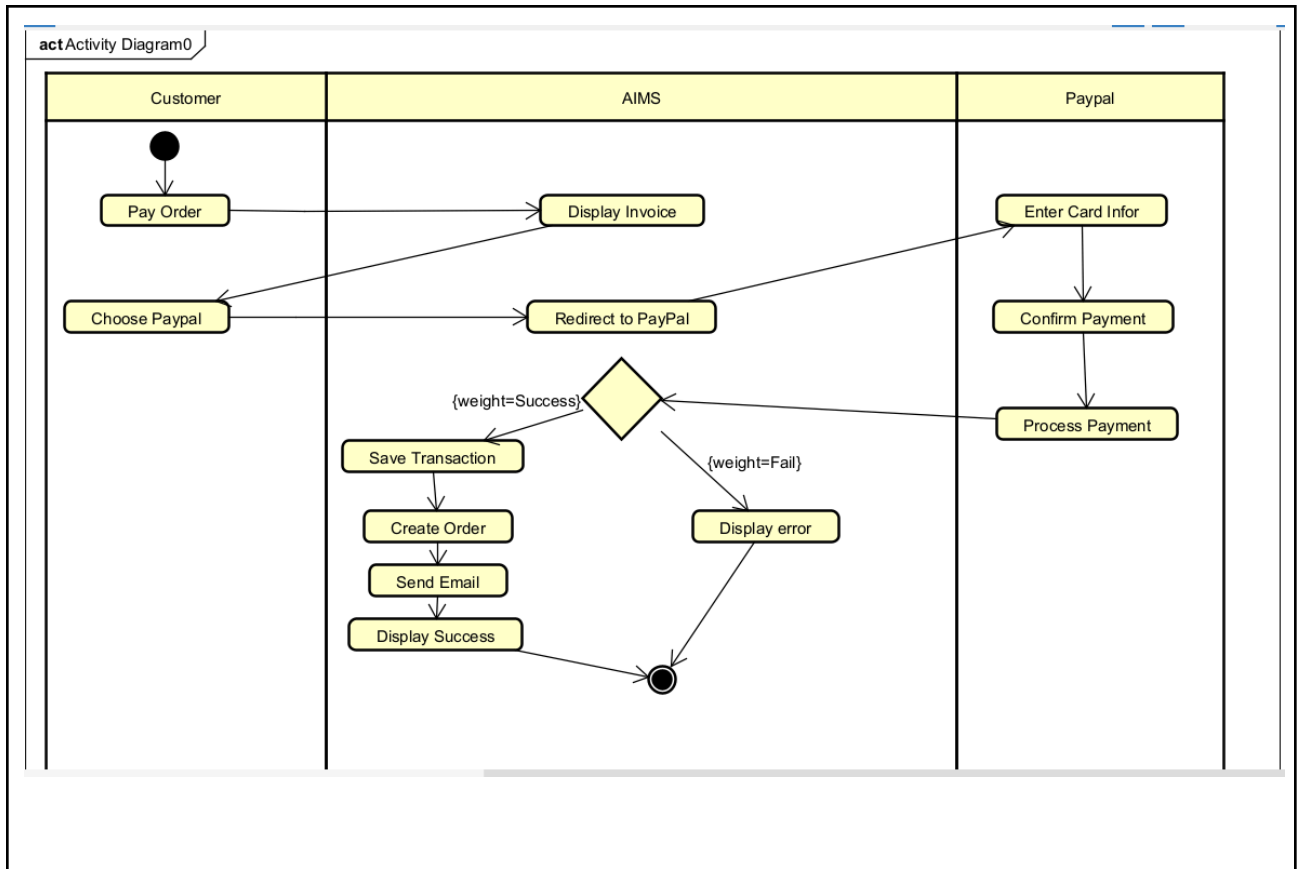
|   |               |                                      |                  |                   |
|---|---------------|--------------------------------------|------------------|-------------------|
| 1 | transactionId | Unique ID of the payment transaction | String           | TXN456789         |
| 2 | status        | Result of the payment transaction    | SUCCESS / FAILED | SUCCESS           |
| 3 | message       | Description of transaction result    | String           | Payment completed |

## 9. Postconditions

If the payment is successful, the order is marked as paid in the system.

The payment result is returned to the Place Order use case.

If the payment fails, the order remains unpaid.





## 4 Supplementary specification

### 4.1 Functionality

The following functional requirements are general to many use cases and apply across the system:

#### **Taxation Policy:**

- A 10% Value-Added Tax (VAT) is applied to all products.
- Both the original value and the current price stored in the system are without VAT.
- The total amount customers pay includes the total product price with VAT plus the delivery fee.
- Shipping fees are not subject to tax.

#### **Pricing Integrity:**

- The current price of a product may change depending on market demand.
- The current price must always be between 30% and 150% of the product's original value to avoid price inflation or undercutting.

#### **Shipping Fee Calculation:**

- Shipping fees are calculated based on the total weight of all items and the delivery location.
- For customers located in Ha Noi or Ho Chi Minh City: the initial price for the first 3 kg is 22,000 VND.
- For customers elsewhere in Vietnam: the initial price for the first 0.5 kg is 30,000 VND.
- An additional fee of 2,500 VND is charged for every subsequent 0.5 kg.
- Orders with a total value of items exceeding 100,000 VND qualify for free shipping, up to a maximum discount of 25,000 VND per order.
- Each time customers enter or update the delivery address, the system will calculate (or recalculate) and display the delivery fee.

#### **Payment:**

- The default payment method is VietQR (QR code-based payment).
- The alternative payment method is credit card through the PayPal Sandbox gateway.
- Other payment gateways may be supported in the future.

- For order cancellation refunds: PayPal supports automatic refunds via API; VietQR requires manual refund by the product manager.
- For rejected order refunds: PayPal processes automatic refunds; VietQR requires the system to notify the product manager for manual refund processing.

#### **Audit Logging:**

- The system stores the history of all product additions, edits, and deletions. Product managers can query these histories at any time.
- Any sensitive administrative actions (such as email updates or password resets) must be logged.
- The system records payment transaction information and successfully paid orders.

#### **Automated Notifications:**

- After successful payment, the system sends invoice and payment transaction information to the customer's email.
- When an order is approved or rejected, a notification email is sent to the customer.
- Sensitive administrative actions are automatically notified to the affected users.
- In the future, additional notification methods such as SMS or push notifications may also be supported.

#### **Validation:**

- The system notifies users if any operation is invalid, for example, when the input does not comply with the rules or the date format is incorrect.
- During delivery information entry, the system checks for required fields left blank or invalid information and asks customers to update.

#### **Product Deletion Rules:**

- Up to 10 products may be deleted simultaneously.
- Managers cannot delete more than 20 products per day.
- A product can only be permanently deleted if its stock is equal to zero; otherwise, its status is changed to "deactivated".
- Managers may add an unlimited number of products.

### **Stock Management:**

- Stock quantity is automatically updated whenever items are sold.
- Manual stock adjustments (e.g., new items added, damaged items removed) must be performed by the product manager, and the reason for the stock change must be explicitly recorded.

### **4.2 Usability**

- No login is required for customers. Customers can browse, search, add to cart, and complete purchases without creating an account.
- The system allows new users to easily familiarize themselves with its features and navigation.
- When starting the software, a list of 20 random products is displayed to provide immediate content.
- Customers can search for products using title or category, and filter by price ranges (e.g., under 100,000 VND, 100,000–200,000 VND, 200,000–300,000 VND, and so on).
- The shipping fee is dynamically recalculated and displayed each time the customer enters or updates the delivery address.
- The system provides clear, actionable error messages when invalid operations are attempted, allowing users to correct and retry.
- Customers can go back to any step or exit the software during the ordering process.
- Customers can view order information or cancel orders using links sent in the email.

### **4.3 Reliability**

- The system shall operate continuously for 300 hours without failure.
- The system shall resume normal operation within a maximum of 1 hour after an incident.
- The system shall be available 24/7 for customer-facing features.
- Transaction integrity shall be maintained on payment gateway failure: the cart and order information shall be preserved, and no duplicate charges shall occur.

### **4.4 Performance**

- The system shall serve up to 1,000 customers simultaneously without significantly reducing performance.
- The maximum response time shall be 2 seconds under normal conditions.
- The maximum response time shall be 5 seconds during peak hours.

#### **4.5 Supportability**

- The system shall support both web and desktop application deployment.
- The payment integration shall be modular and loosely coupled, allowing future addition of new payment gateways (e.g., Momo, ZaloPay) without major architectural changes.
- The product catalog shall be designed with an extensible schema (e.g., inheritance-based pattern) to support new product types in the future.
- Audit logs shall include manager IDs, timestamps, and change details for system maintenance and troubleshooting.

#### **4.6 Other requirements**

##### ***Security:***

- Role-Based Access Control (RBAC) shall be implemented for the administration portal. Administrators and product managers must log in to access features corresponding to their roles.
- All user passwords shall be securely hashed. Administrators may trigger password reset processes but cannot access users' actual passwords.
- Any sensitive administrative actions (such as email updates or password resets) must be logged and automatically notified to the affected users.
- User account management shall follow the principle of least privilege and data protection standards.

##### **External Dependencies:**

- The system depends on the PayPal Sandbox API for credit card payment processing and refunds.
- The system depends on the VietQR API for QR code-based payment processing.
- Availability and performance of the system's payment features are contingent on the availability of these external APIs.